Python Programming
Unit 06 – Lecture 01 Notes
NumPy Basics (ndarray, Operations, Broadcasting)

Tofik Ali

February 20, 2026

Contents

1 Lecture Overview

NumPy is the core library for numerical computing in Python. It provides **ndarray** objects (n-dimensional arrays) and fast vectorized operations. This lecture focuses on:

- array creation,
- array attributes,
- vectorized operations,
- and broadcasting.

2 Setup (If Needed)

If NumPy is not installed:

```
pip install numpy
```

3 Core Concepts

3.1 List vs NumPy Array

Python lists can store mixed types and are not optimized for numerical math. NumPy arrays store values in a compact block of memory and use fixed `dtype`.

```python
import numpy as np

lst = [1, 2, 3]
arr = np.array([1, 2, 3])

print(lst * 2) # [1, 2, 3, 1, 2, 3]
print(arr * 2) # [2 4 6]
```

3.2 Array Creation

Common creation functions:

- `np.array([...])`
- `np.zeros((r,c))`
- `np.ones((r,c))`
- `np.arange(start, stop, step)`
- `np.linspace(start, stop, num)`

```python
a = np.zeros((2, 3))
b = np.ones((2, 3))
c = np.arange(0, 10, 2)
d = np.linspace(0, 1, 5)
```

3.3 Array Attributes

Useful attributes:

- `shape`: tuple of dimensions
- `ndim`: number of dimensions
- `size`: total number of elements
- `dtype`: data type of elements

```
m = np.array([[1, 2], [3, 4]])
print(m.shape) # (2, 2)
print(m.ndim) # 2
print(m.size) # 4
print(m.dtype) # int64 (or similar)
```

3.4 Vectorized Operations

NumPy applies operations element-wise:

```
x = np.array([1, 2, 3])
print(x + 10) # [11 12 13]
print(x * x) # [1 4 9]
```

3.5 Broadcasting (Beginner View)

Broadcasting allows operations between arrays of different shapes when compatible. The simplest forms:

- array and scalar (scalar applied to all elements)
- matrix and vector (vector applied across rows/columns depending on shape)

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
v = np.array([10, 20, 30])
print(a + v)
```

4 Demo Walkthrough

File: `demo/numpy_basics_demo.py`
Observe:

- creating arrays using multiple methods,
- applying vectorized operations,
- broadcasting with a row vector.

5 Interactive Checkpoints (with Solutions)

Checkpoint 1 Solution

Question: why is `[1,2,3]*2` different?
Answer: list multiplication repeats the list, while NumPy array multiplication is numeric and element-wise.

Checkpoint 2 Solution

Question: what is broadcasting?
Answer: a rule system that expands smaller shapes to match larger shapes for element-wise operations.

6 Practice Exercises (with Solutions)

Exercise 1: Create a 3x3 Array of Ones

Solution:

```
import numpy as np
a = np.ones((3, 3))
print(a)
```

Exercise 2: Square All Elements

Solution:

```
import numpy as np
x = np.array([1, 2, 3, 4])
print(x * x)
```

7 Exit Question (with Solution)

Question: name two array creation functions.
Answer: `np.array`, `np.zeros` (many answers).

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Repository: `https://github.com/tali7c/Python-Programming`

Quick Links

Core Concepts **Demo** **Interactive** **Summary**

Agenda

- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Explain why NumPy is used for numerical computing
- Create arrays and understand `dtype` and shape
- Apply vectorized operations on arrays
- Explain broadcasting with examples

Why NumPy?

- Fast numerical computing (implemented in C)
- Compact arrays with fixed data types
- Vectorized operations (no Python loops for many tasks)
- Foundation for Pandas, ML, and scientific computing

List vs NumPy Array

- List: can store mixed types, slower for numeric math
- NumPy array: fixed dtype, faster operations

```
import numpy as np
x = np.array([1, 2, 3])
print(x * 2) # [2 4 6]
```

Array Creation

- `np.array([...])`
- `np.zeros((r,c))`, `np.ones((r,c))`
- `np.arange(start, stop, step)`
- `np.linspace(start, stop, num)`

Array Attributes

- `shape`, `ndim`, `size`, `dtype`

```
a = np.array([[1, 2], [3, 4]])
print(a.shape, a.ndim, a.size, a.dtype)
```

Broadcasting (Idea)

- NumPy can apply operations between arrays of different shapes
- Example: add a scalar to every element

```
a = np.array([1, 2, 3])
print(a + 10) # [11 12 13]
```

Demo: Array Creation + Broadcasting

- File: `demo/numpy_basics_demo.py`
- Shows:
  - creating arrays
  - vectorized math
  - broadcasting examples

Checkpoint 1

Question: Why is `[1,2,3] * 2` different from `np.array([1,2,3]) * 2`?

Checkpoint 2

Question: What does broadcasting mean in NumPy?

Think-Pair-Share

Discuss:

- When would you still use a Python list instead of a NumPy array?

Key Takeaways

- NumPy arrays are fast and have fixed `dtype`
- Vectorized operations avoid explicit loops
- Broadcasting applies operations across compatible shapes

Exit Question

Name any two NumPy functions used to create arrays.